MP7: Simple File System

Mohammed Sharique
UIN: 837004870
CSCE611: Operating System

May 4, 2026

Assigned Tasks

Main: Completed.
Bonus 1: Completed.
Bonus 2: Completed.

System Design

The goal of this machine problem was to implement a simple, block-based file system supporting sequential file access, with files identified by unsigned integer IDs and no directory hierarchy. The file system is realized through the FileSystem and File classes, operating directly on a block device abstraction. The design emphasizes simplicity, correctness, and extensibility for larger file sizes.

- I designed the FileSystem class to manage all on-disk metadata, including the inode table and the free block bitmap. The file system is initialized with a fixed size (specified at format time), and all allocations and metadata structures are sized accordingly to ensure only the formatted region of the disk is used.
- Each file is represented by an Inode structure, which stores the file's unique ID, the block number of its index block (for indirect addressing), and the current file size in bytes. The index block contains an array of block numbers, allowing each file to span multiple disk blocks (up to 64KB per file).
- The Format operation initializes the inode table and free block list on disk, marking metadata blocks as used. The Mount operation loads these structures into memory, using the size parameter to determine the number of usable blocks, thus decoupling the file system's logical size from the physical disk size.
- File creation (CreateFile) allocates a free inode, an index block, and the first data block, updating all metadata structures and writing initial values to disk. File deletion (DeleteFile) traverses the index block, frees all associated data blocks, and clears the inode, ensuring no space leaks.
- The File class provides sequential read and write operations. Upon opening a file, all data blocks referenced by the index block are loaded into memory for efficient access. Reads and writes update the in-memory cache and the file size as needed, allocating new blocks only when writing past the current end of file. The destructor ensures all modified blocks and metadata are flushed back to disk.
- All block allocations and deallocations are managed through the free block bitmap, and all metadata updates are synchronized to disk via the sync_metadata() method to ensure consistency.
- The design supports files up to 64KB by using a single-level indirect block (index block) per file, with each index block able to reference up to 128 data blocks (with 512B block size). This approach balances simplicity and scalability for the assignment's requirements.

Bonus 1: Supporting 64kB Files in the File System

To extend the basic file system to support files up to 64kB, the design transitions from a single-block-per-file model to an inode-based indirect addressing scheme. This enables each file to span up to 128 data blocks, thus supporting files as large as 64kB. The following details the required data structure changes, allocation logic, and the precise modifications to file operations.

- **Inode Structure and Index Block:** Each file's inode is extended to include a pointer to an *index block* (also called an indirect block). The index block is a 512B block containing an array of 128 integers, each storing the block number of a data block belonging to the file. The inode also tracks the file's size in bytes for efficient end-of-file and boundary checks.
- **File Allocation Changes:**
  - **CreateFile:** When creating a file, the system allocates a free inode, a free index block, and the first data block. The index block is initialized with all entries set to `-1`, except the first entry, which is set to the first data block's number. The inode is updated to point to the index block, and all allocations are marked in the free block bitmap.
  - **DeleteFile:** To delete a file, the system reads the index block, iterates through all valid entries, and frees each data block. The index block itself is then freed, and the inode is cleared. All changes are synchronized to disk to prevent space leaks.
  - **Metadata Management:** The free block bitmap and inode table are updated on disk after every allocation or deallocation to maintain consistency.
- **Data Structure Changes:**
  - The `Inode` struct/class now contains an `index_block_no` field (block number of the index block) and a `size` field (file size in bytes).
  - The index block is a new on-disk structure: an array of 128 integers, each referencing a data block.
  - In the `File` class, the index block and all file blocks are loaded into heap-allocated arrays (`index_block_cache` and `file_block_caches`) for efficient sequential access.
- **Read and Write Operation Changes in `File`:**
  - **Read:** The `Read` method uses the current file position to compute the block index and offset. It reads data from the corresponding cached block, advancing the position and handling block boundaries. Reads never go past the file's logical end (`inode->size`).
  - **Write:** The `Write` method similarly computes the target block and offset. If writing past the current allocation, it allocates a new data block, updates the index block and free block bitmap, and initializes the new block. The file size is updated as needed. If the file is truncated (overwritten with less data), an EOF marker is written at the new end.
  - **Constructor/Destructor:** The constructor loads the index block and all referenced data blocks into memory. The destructor writes back any modified blocks and the index block, ensuring all changes are persisted.
- **Functions Added/Modified:**
  - `CreateFile`, `DeleteFile` in `FileSystem` are modified to handle index block allocation and deallocation.
  - The `File` class constructor, destructor, `Read`, and `Write` methods are updated to support multi-block files via the index block.
  - No changes are needed for `Reset` and `EoF`, as they operate on the logical file position and size.

Code Description

I modified the files file.C/H, file_system.C/H and kernel.C (to modify exercise_file_system to test 64kB files).
The code compiles by running make in the project directory.
Github: https://github.com/shariqu3/csce-611-machine-problems/tree/v2

file_system.C:FileSystem::FileSystem This constructor initializes the `FileSystem` object, setting up the internal disk pointer, calculating the number of blocks (`n_blocks`) based on the provided size, and allocating memory for the `free_blocks` bitmap and the `inodes` array. It ensures the file system is ready for mounting or formatting operations.

```
FileSystem::FileSystem(unsigned int _size) {
    // Console::puts("In file system constructor.\n");
    disk = NULL;
    // n_blocks = MAX_INODES;
    n_blocks = _size / SimpleDisk::BLOCK_SIZE;
    size = n_blocks* SimpleDisk::BLOCK_SIZE;
    free_blocks = new unsigned char[n_blocks];
    inodes = new Inode[n_blocks];
}
```

file_system.C:FileSystem::~FileSystem The destructor ensures a clean shutdown of the file system. It prints a status message, synchronizes all metadata to disk using `sync_metadata()`, and then deallocates the memory used for the `free_blocks` and `inodes` arrays, preventing memory leaks.

```
FileSystem::~FileSystem() {
    Console::puts("unmounting file system\n");
    /* Make sure that the inode list and the free list are saved. */
    sync_metadata();
    delete[] free_blocks;
    delete[] inodes;
}
```

file_system.C:Mount This function mounts a disk by reading the inode list and free block bitmap from disk into memory. It first checks if a disk is already mounted, initializes internal pointers, and loads metadata from the first two blocks of the disk, preparing the file system for use.

file_system.C:Format This static function formats the disk by initializing the inode list and free block bitmap. It writes empty inodes and marks the metadata blocks as used, ensuring a clean state for new file system operations.

file_system.C:sync_metadata This function writes the current in-memory inode list and free block bitmap back to disk, ensuring that all metadata changes are persisted and the file system remains consistent across reboots or unmounts.

file_system.C:LookupFile This function searches the inode list for a file with a given identifier. It iterates through all inodes, returning a pointer to the matching inode if found, or `nullptr` if the file does not exist.

file_system.C:CreateFile This function creates a new file by allocating a free inode, an index block, and a data block. It initializes the inode metadata, updates the index block, marks the relevant blocks as used, and synchronizes the metadata to disk. If the file already exists or resources are exhausted, it reports an error.

```cpp
bool FileSystem::Mount(SimpleDisk * _disk) {
    Console::puts("mounting file system from disk\n");
    /* Here you read the inode list and the free list into memory */
    /*
    1. Check if disk is NULL, if not return false
    2. Initialize  disk, n_blocks & size
    3. Read to memory inodes and free_blocks
    End: return true
    */

    // 1. Check if disk is NULL, if not return false
    if(disk != NULL)
        return false;

    // 2. Initialize  disk, n_blocks & size
    disk = _disk;

    // 3. Read to memory inodes and free_blocks
    disk->read(0, (unsigned char* )inodes);

    unsigned char* buf = new unsigned char[SimpleDisk::BLOCK_SIZE/sizeof(unsigned char)];
    disk->read(1, buf);
    for(unsigned int i=0;i<n_blocks;i++)
        {free_blocks[i] = buf[i];}

    return true;
}
```

```cpp
bool FileSystem::Format(SimpleDisk * _disk, unsigned int _size) { // static!
    Console::puts("formatting disk\n");

    /* Here you populate the disk with an initialized (probably empty) inode list
       and a free list. Make sure that blocks used for the inodes and for the free list
       are marked as used, otherwise they may get overwritten. */
    /*
    1. Overwrite inodes block
    2. Overwrite free_block
    */

    // 1. Overwrite inodes block
    struct Inode* buf = new Inode[MAX_INODES];
    _disk->write(0, (unsigned char*)buf);
    delete[] buf;

    // 2. Overwrite free_block
    unsigned char* buf2 = new unsigned char[MAX_INODES];
    buf2[0] = 1;
    buf2[1] = 1;
    _disk->write(1, buf2);
    delete[] buf2;
    return true;
}
```

```cpp
void FileSystem::sync_metadata()
{
    disk->write(0, (unsigned char *) inodes);
    disk->write(1, free_blocks);
}
```

```cpp
Inode * FileSystem::LookupFile(int _file_id) {
    Console::puts("looking up file with id = "); Console::puti(_file_id); Console::puts("\n");
    /* Here you go through the inode list to find the file. */
    for(unsigned int i=0;i<n_blocks;i++)
    {
        Inode * inode = &inodes[i];
        if(inode->id == _file_id)
        {
            return inode;
        }
    }
    return nullptr;
}
```

file_system.C:DeleteFile This function deletes a file by locating its inode, freeing all associated data blocks and the index block, and invalidating the inode. It updates the free block bitmap accordingly, ensuring that the space can be reused for future files.

```cpp
bool FileSystem::CreateFile(int _file_id) {
    Console::puts("creating file with id:"); Console::puti(_file_id); Console::puts("\n");
    /* Here you check if the file exists already. If so, throw an error.
       Then get yourself a free inode and initialize all the data needed for the
       new file. After this function there will be a new file on disk. */
    /*
    1. Lookup if file exists
    2. Find a free block
    3. Find a free inode
    4. Set file metadata on inode
    5. Allocate index_block for inode
    6. Allocate first file block
    7. Mark the blocks as reserved
    8. Rewrite block with first char as end of file(-1);
    9. Update inode list and freelist on disk
    */

    // 1. Lookup if file exists
    Inode* ret = LookupFile(_file_id);

    if(ret)
    {
        Console::puts("File already exists! File ID:");
        Console::puti(_file_id);
        return false;
    }

    // 2. Find free blocks &
    // 5. Allocate index_block for inode &
    // 6. Allocate first file block &
    // 7. Mark the blocks as reserved
    int index_block_no = GetFreeBlock();
    if(index_block_no == -1)
    {
        Console::puts("Disk Space Full!");
        assert(false);
    }
    free_blocks[index_block_no] = 1;

    int file_block_no = GetFreeBlock();
    if(file_block_no== -1)
    {
    free_blocks[index_block_no] = 0;
        Console::puts("Disk Space Full!");
        assert(false);
    }
    free_blocks[file_block_no] = 1;

    // 3. Find a free inode
    short inode_idx= GetFreeInode();

    if(inode_idx== -1)
    {
    free_blocks[index_block_no] = 0;
    free_blocks[file_block_no] = 0;
        Console::puts("Disk Space Full!");
        assert(false);
    }
```

file_system.C:GetFreeInode This function scans the inode list for an unused entry (where `id == -1`) and returns its index. If no free inode is available, it returns `-1`, signaling that the file system is full.

file_system.C:GetFreeBlock This function searches the free block bitmap for an available block (where the value is not `1`) and returns its index. If no free block is found, it returns `-1`, indicating that the disk is full.

file.C:File::File This constructor opens a file by locating its inode using the provided file identifier. It initializes the file's internal position pointers, reads the index block into a cache, and loads all associated data blocks into memory for efficient sequential access. If the file does not exist, it reports an error and aborts.

file.C:File::~File The destructor ensures all cached file data is written back to disk. It iterates through all loaded blocks, writing any changes, and updates the index block. It also synchronizes the inode metadata to disk and deallocates all dynamically allocated memory, ensuring data integrity and preventing memory leaks.

```
    // 4. Set file metadata on inode
    struct Inode * inode = &inodes[inode_idx];
    inode->index_block_no = index_block_no;
    inode->id = _file_id;
    inode->size = 0;


    // 6. Rewrite block with first char as end of file(-1);
    unsigned char * buf = new unsigned char[SimpleDisk::BLOCK_SIZE];
    buf[0] = -1;
    disk->write(file_block_no, buf);
    delete[] buf;

    int *index_block_buf = new int[SimpleDisk::BLOCK_SIZE/sizeof(int)];
    for(unsigned int i=0;i<SimpleDisk::BLOCK_SIZE/sizeof(int);i++)
    {
        index_block_buf[i] = -1;
    }
    index_block_buf[0] = file_block_no;
    disk->write(index_block_no, (unsigned char *)index_block_buf);
    delete[] index_block_buf;

    //7. Update inode list and freelist on disk
    sync_metadata();
    return true;
}
```

```
bool FileSystem::DeleteFile(int _file_id) {
    Console::puts("deleting file with id:"); Console::puti(_file_id); Console::puts("\n");
    /* First, check if the file exists. If not, throw an error.
       Then free all blocks that belong to the file and delete/invalidate
       (depending on your implementation of the inode list) the inode. */
    /*
    1. Find inode
    2. Read index block
    3. Mark all file blocks as free
    4. Mark index block as free
    5. Make inode empty
    */

    // 1. Find inode
    Inode *inode= LookupFile(_file_id);
    if(inode==nullptr)
    {
        Console::puts("File does not exist!");
        assert(false);
    }

    // 2. Read index block
    int index_block_no = inode->index_block_no;
    unsigned char * buf = new unsigned char[SimpleDisk::BLOCK_SIZE];
    disk->read(index_block_no, buf);

    // 3. Mark all file blocks as free
    int *block_lists = (int *)buf;
    for(unsigned int i=0;i<SimpleDisk::BLOCK_SIZE/sizeof(int);i++)
    {
        if(block_lists[i] != -1)
        {
            free_blocks[block_lists[i]] = 0;
        }
    }

    // 4. Mark index block as free
    free_blocks[index_block_no] = 0;

    // 5. Make inode empty
    inode->index_block_no = -1;
    inode->id = -1;
    inode->size = 0;
    return true;
}
```

file.C:File::Read This function reads up to _n bytes from the file into the provided buffer, starting at the current file position. It fetches data from the cached blocks, updates the current position, and stops at either the requested byte count or the end of the file.

```
short FileSystem::GetFreeInode(){
    for(short int i=0;i<(short)n_blocks;i++)
    {
        if(inodes[i].id == -1)
        {
            return i;
        }
    }
    return -1;
}
```

```
int FileSystem::GetFreeBlock(){
    for(int i=0;i<n_blocks;i++)
    {
        if(free_blocks[i] != 1)
        {
            return i;
        }
    }
    return -1;
}
```

```
File::File(FileSystem *_fs, int _id) {
    Console::puts("Opening file.\n");
    current_pos = 0;
    fs = _fs;
    inode = fs->LookupFile(_id);
    if(inode == NULL)
    {
        Console::puts("File Not Found!");
        assert(false);
    }
    // Reading the index block to cache
    index_block_cache = new int[SimpleDisk::BLOCK_SIZE/sizeof(int)];
    fs->disk->read(inode->index_block_no, (unsigned char *)index_block_cache);

    // Reading all file blocks to cache
    file_block_caches = new unsigned char*[SimpleDisk::BLOCK_SIZE / sizeof(int)];
    n_loaded_blocks = SimpleDisk::BLOCK_SIZE / sizeof(int);
    for(int i = 0; i < n_loaded_blocks; ++i){
        file_block_caches[i] = new unsigned char[SimpleDisk::BLOCK_SIZE];
        if(index_block_cache[i] != -1) {
            fs->disk->read(index_block_cache[i], file_block_caches[i]);
        }
    }

    current_pos = 0;
    current_block_idx = 0;
    current_offset = 0;
}
```

file.C:File::Write This function writes up to `_n` bytes from the input buffer into the file, allocating new blocks as needed. It updates the inode size if the file grows, marks new blocks as used, and ensures the `EOF` marker is set if the file shrinks. The current file position is advanced accordingly.

```cpp
File::~File() {
    Console::puts("Closing file.\n");
    /* Make sure that you write any cached data to disk. */
    /* Also make sure that the inode in the inode list is updated. */
    for(int i = 0; i < n_loaded_blocks; ++i) {
        if(index_block_cache[i] != -1) {
            fs->disk->write(index_block_cache[i], file_block_caches[i]);
        }
        delete[] file_block_caches[i];
    }
    delete[] file_block_caches;

    // Write back index block cache
    fs->disk->write(inode->index_block_no, (unsigned char*)index_block_cache);
    delete[] index_block_cache;

    fs->sync_metadata();
}
```

```cpp
int File::Read(unsigned int _n, char *_buf) {
    Console::puts("reading from file\n");
    int n_read = 0;
    while(n_read < (int)_n && current_pos < inode->size) {
        int block_idx = current_pos / SimpleDisk::BLOCK_SIZE;
        int offset = current_pos % SimpleDisk::BLOCK_SIZE;
        if(index_block_cache[block_idx] == -1)
        { break; }

        _buf[n_read++] = file_block_caches[block_idx][offset];
        ++current_pos;
    }
    current_block_idx = current_pos / SimpleDisk::BLOCK_SIZE;
    current_offset = current_pos % SimpleDisk::BLOCK_SIZE;
    return n_read;
}
```

```cpp
int File::Write(unsigned int _n, const char *_buf) {
    Console::puts("writing to file\n");
    int old_size = inode->size;
    int n_written = 0;
    while(n_written < (int)_n && current_pos < (SimpleDisk::BLOCK_SIZE * n_loaded_blocks)) {
        int block_idx = current_pos / SimpleDisk::BLOCK_SIZE;
        int offset = current_pos % SimpleDisk::BLOCK_SIZE;

        // Allocate new block if needed
        if(index_block_cache[block_idx] == -1) {
            int new_block = fs->GetFreeBlock();
            if(new_block == -1) {
                Console::puts("Disk Space Full!");
                assert(false);
            }
            index_block_cache[block_idx] = new_block;
            fs->free_blocks[new_block] = 1;
            file_block_caches[block_idx] = new unsigned char[SimpleDisk::BLOCK_SIZE];
        }

        file_block_caches[block_idx][offset] = _buf[n_written++];
        current_pos++;
        if(current_pos > inode->size)
        {
            inode->size = current_pos;
        }
    }
    current_block_idx = current_pos / SimpleDisk::BLOCK_SIZE;
    current_offset = current_pos % (SimpleDisk::BLOCK_SIZE);
    if (inode->size < old_size) {
        int eof_block = inode->size / SimpleDisk::BLOCK_SIZE;
        int eof_offset = inode->size % SimpleDisk::BLOCK_SIZE;
        if (eof_block < n_loaded_blocks) {
            file_block_caches[eof_block][eof_offset] = (unsigned char)0xFF; // EOF marker
        }
    }
    return n_written;

}
```

file.C:File::Reset This function resets the file's internal position pointers to the beginning, allowing subsequent read or write operations to start from the start of the file.

```
void File::Reset() {
    Console::puts("resetting file\n");
    current_pos = 0;
    current_block_idx = 0;
    current_offset = 0;
}
```

file.C:File::EoF This function checks if the current file position has reached or exceeded the file's size, indicating the end-of-file condition for sequential operations.

```
bool File::EoF() {
    Console::puts("checking for EoF\n");
    return current_pos >= inode->size;
}
```

kernel.C:exercise_file_system : This function serves as the main test harness for validating the file system implementation. It was modified to accept an additional file_size parameter, enabling flexible testing with files of arbitrary size (up to 64 KB in our experiments). The function allocates two buffers, fills them with distinct patterns, and then exercises the file system by creating two files, writing the buffers to them, closing and reopening the files, and verifying that the contents are read back correctly. After successful verification, both files are deleted and all allocated memory is released. This comprehensive test ensures correct file creation, data integrity across open/close cycles, and proper resource cleanup for files of varying sizes.

file_system.C:Inode::Inode : This constructor initializes a new Inode object to a default, unused state. It sets the id field to -1 to indicate that the inode is not currently assigned to any file, and sets index_block_no to -1 to show that no index block is allocated. This ensures that all newly created inodes are clearly marked as free and ready for allocation by the file system.

Testing

For this machine problem, I used the provided exercise_file_system function as the primary test harness to validate the file system implementation. To further stress-test the system, I modified exercise_file_system to accept a file_size parameter, allowing for flexible testing with files of varying sizes. The test suite was executed with file sizes up to 64 KB, ensuring that the file system could correctly handle file creation, writing, reading, and deletion for large files spanning multiple disk blocks.

During testing, I closely monitored the serial output for correct file content verification, successful allocation and deallocation of disk blocks, and proper synchronization of metadata. The tests confirmed that the file system maintained data integrity across repeated open/close cycles and that all resources were properly reclaimed after file deletion.

```cpp
void exercise_file_system(FileSystem* _file_system, unsigned int _iteration_no, unsigned int file_size=64 KB) {

    char* buf1 = new char[file_size];
    char* buf2 = new char[file_size];

    // Fill buffers with patterns
    for (unsigned int i = 0; i < file_size; ++i) {
        buf1[i] = (char)((_iteration_no + i) % 256); // pattern 1
        buf2[i] = (char)((_iteration_no + 128 + i) % 256);// pattern 2
    }

    Console::puts("Creating File 1 and File 2\n");
    assert(_file_system->CreateFile(1));
    assert(_file_system->CreateFile(2));

    {
        Console::puts("Opening File 1 and File 2\n");
        File file1(_file_system, 1);
        File file2(_file_system, 2);

        Console::puts("Writing into File 1 and File 2\n");
        assert(file1.Write(file_size, buf1) == (int)file_size);
        assert(file2.Write(file_size, buf2) == (int)file_size);

        Console::puts("Closing File 1 and File 2\n");
    }

    {
        Console::puts("Opening File 1 and File 2 again\n");
        File file1(_file_system, 1);
        File file2(_file_system, 2);

        Console::puts("Checking content of File 1 and File 2\n");

        file1.Reset();
        char* result1 = new char[file_size];
        assert(file1.Read(file_size, result1) == (int)file_size);
        for (unsigned int i = 0; i < file_size; i++) {
            assert(result1[i] == buf1[i]);
        }
        delete[] result1;

        file2.Reset();
        char* result2 = new char[file_size];
        assert(file2.Read(file_size, result2) == (int)file_size);
        for (unsigned int i = 0; i < file_size; i++) {
            assert(result2[i] == buf2[i]);
        }
        delete[] result2;

        Console::puts("SUCCESS!!\n");
        Console::puts("Closing File 1 and File 2 again\n");
    }

    Console::puts("Deleting File 1 and File 2\n");
    assert(_file_system->DeleteFile(1));
    assert(_file_system->LookupFile(1) == nullptr);
    assert(_file_system->DeleteFile(2));
    assert(_file_system->LookupFile(2) == nullptr);

    delete[] buf1;
    delete[] buf2;
}
```

```cpp
Inode::Inode(){
    id = -1;
    index_block_no=-1;
}
```